Java w IntelliJ IDEA - obiekty i dziedziczenie

BLOK 6 - static, final, zasięg zmiennych i klasy potomne

1. Klasa i obiekt - co należy do definicji, a co do egzemplarza?

Klasa jest opisem typu, a obiekt konkretną instancją utworzoną na podstawie tego opisu. Klasa `Car` może przewidywać pola `brand` i `mileage`, a każdy samochód ma własne wartości tych pól. Jeżeli zmienimy przebieg jednego obiektu, przebieg drugiego nie zmieni się automatycznie.

```
class Car {
    String brand;
    int mileage;
}

Car first = new Car();
Car second = new Car();
first.brand = "Ford";
second.brand = "Toyota";
System.out.println(first.brand);
System.out.println(second.brand);
```

```
Wynik:
Ford
Toyota
```

Zmienna `first` odnosi się do pierwszego obiektu, a `second` do drugiego. W obu obiektach występuje pole `brand`, ale każde jest częścią stanu własnej instancji. Ten rodzaj pola nazywamy polem instancji.

2. Pole static - jedna wartość wspólna dla całej klasy

Pole oznaczone `static` nie jest przechowywane osobno dla każdego obiektu. Należy do klasy i może być używane do przechowywania wartości wspólnych, np. licznika utworzonych obiektów. Typowy zapis odwołania to `ClassName.fieldName`, dzięki któremu widzimy, że chodzi o składnik klasy.

```
class Visitor {
    static int numberOfVisitors = 0;
    Visitor() { numberOfVisitors++; }
}

new Visitor();
new Visitor();
System.out.println(Visitor.numberOfVisitors);
```

```
Wynik:
2
```

Pierwsze utworzenie obiektu zwiększa wspólny licznik z 0 na 1, drugie z 1 na 2. Gdyby pole nie było statyczne, każdy nowy obiekt otrzymywałby własny licznik. Nie należy jednak umieszczać w polach statycznych danych, które powinny należeć wyłącznie do jednego użytkownika lub jednej instancji.

3. Metody static i metody instancji

Metodę statyczną można wywołać bez tworzenia obiektu. Metoda instancji działa w kontekście określonego obiektu i może bezpośrednio odczytywać jego pola. Stąd `Math.max` jest metodą statyczną, ale metoda `deposit` konkretnego konta bankowego powinna zazwyczaj być metodą instancji.

```
class Counter {
    int value = 0;
    void increase() { value++; }
    static int doubled(int x) { return x * 2; }
}

Counter counter = new Counter();
counter.increase();
System.out.println(counter.value);
System.out.println(Counter.doubled(4));
```

```
Wynik:
1
8
```

`counter.increase()` działa na polu konkretnego obiektu. `Counter.doubled(4)` nie potrzebuje obiektu, bo korzysta wyłącznie z argumentu. Metoda statyczna nie może po prostu użyć `this`, ponieważ nie jest związana z aktualną instancją.

4. final przy zmiennych - czego już nie można zrobić?

Słowo `final` oznacza, że po przypisaniu wartości nie można przypisać tej samej zmiennej nowej wartości. W przypadku typów prostych oznacza to brak możliwości zastąpienia liczby lub wartości logicznej. Przy referencji do obiektu `final` zabrania zmiany samej referencji, ale niekoniecznie zmiany wnętrza obiektu.

```
final int MAX_ATTEMPTS = 3;
System.out.println(MAX_ATTEMPTS);
// MAX_ATTEMPTS = 4; // błąd kompilacji
```

Stałe często zapisujemy wielkimi literami z podkreśleniami, zwłaszcza gdy są polami `static final`. Jest to umowa dotycząca czytelności kodu, a nie wymaganie języka. Sama nazwa nie zmienia zachowania zmiennej; decyduje słowo kluczowe `final`.

5. final przy obiekcie nie oznacza niezmienności obiektu

Jeżeli zmienna referencyjna jest `final`, nie możemy sprawić, aby zaczęła wskazywać na nowy obiekt. Możemy jednak nadal modyfikować dostępne pola lub wywoływać metody modyfikujące obiekt, o ile jego klasa na to pozwala.

```
final StringBuilder builder = new StringBuilder("A");
builder.append("B");
System.out.println(builder.toString());
// builder = new StringBuilder("C"); // niedozwolone
```

```
Wynik:
AB
```

Metoda `append` zmieniła zawartość istniejącego obiektu `StringBuilder`. Zmienna `builder` cały czas wskazuje na ten sam obiekt. Nie należy mylić `final` z całkowitą niezmiennością wszystkich danych powiązanych z obiektem.

Składnik	Do czego należy?	Przykład

pole instancji	do konkretnego obiektu	`car.mileage`
pole `static`	do klasy	`Counter.created`
zmienna lokalna	do danego bloku kodu	`int sum = 0;`
parametr	do wywołania metody	`add(int points)`

6. Zmienne lokalne, parametry i pola

Zmienna lokalna jest deklarowana wewnątrz metody lub bloku i jest dostępna wyłącznie w odpowiednim zasięgu. **Parametr** jest zmienną otrzymującą wartość argumentu wywołania. **Pole** należy do klasy lub obiektu. Umiejętność rozróżnienia tych trzech przypadków pomaga zrozumieć, dlaczego jedna zmienna „znika”, a inna pozostaje dostępna.

```
class Score {
    int total = 0;          // pole obiektu
    void add(int points) {  // points - parametr
        int bonus = 2;      // zmienna lokalna
        total += points + bonus;
    }
}
```

Pole `total` nadal istnieje po zakończeniu metody, dopóki istnieje dostępny obiekt. Parametr `points` i lokalna zmienna `bonus` należą do danego wywołania. Po opuszczeniu metody nie możemy odwoływać się do nich z zewnętrznego kodu.

7. Zasięg blokowy - klamry mają znaczenie

Zmienna lokalna jest widoczna od deklaracji do końca odpowiedniego bloku. Jeżeli zostanie zadeklarowana wewnątrz `if` lub pętli, nie można odwołać się do niej poza zakresem jej deklaracji. Java wymaga także, by zmienne lokalne zostały prawidłowo zainicjalizowane przed użyciem.

```
if (true) {
    int temp = 15;
    System.out.println(temp);
}
// System.out.println(temp); // błąd: brak zmiennej w zasięgu
```

```
Wynik:
15
```

To, że wartość 15 była kiedyś wyświetlona, nie sprawia, że nazwa `temp` staje się globalnie dostępna. Zasięg dotyczy miejsca w kodzie, gdzie wolno użyć nazwy. Nie jest tym samym co czas, przez jaki obiekt zajmuje pamięć.

8. this i przysłanianie nazw

Gdy parametr ma taką samą nazwę jak pole klasy, może je przysłonić. W takiej sytuacji piszemy `this.name`, aby wyraźnie wskazać pole aktualnego obiektu. Samo `name` odwołuje się wtedy do parametru o tej nazwie.

```
class Animal {
    private String name;
    Animal(String name) {
        this.name = name;
    }
}
```

Instrukcja `this.name = name` pobiera wartość z argumentu konstruktora i zapisuje ją do pola. Gdyby pomylić strony przypisania lub napisać `name = name`, pole mogłoby pozostać z wartością domyślną `null`, co później prowadziłoby do trudnego w znalezieniu błędu.

9. Dziedziczenie - klasa potomna rozszerza klasę bazową

Dziedziczenie pozwala utworzyć klasę bardziej szczegółową na podstawie innej klasy. W Javie zapisujemy to za pomocą słowa `extends`. Na przykład `Dog` może rozszerzać klasę `Animal`. Klasa potomna dziedziczy dostępne zachowania klasy bazowej, a także może dodawać własne pola i metody.

```
class Animal {
    void breathe() {
        System.out.println("Oddycha");
    }
}
class Dog extends Animal {
    void bark() {
        System.out.println("Hau!");
    }
}

Dog dog = new Dog();
dog.breathe();
dog.bark();
```

```
Wynik:
Oddycha
Hau!
```

Nie napisaliśmy metody `breathe` bezpośrednio w klasie `Dog`, ale obiekt psa może ją wywołać, ponieważ odziedziczył ją po `Animal`. Dziedziczenie ma sens, gdy zachodzi relacja „jest rodzajem”, np. pies jest zwierzęciem. Nie należy go używać wyłącznie dlatego, że dwie klasy mają przypadkiem podobne pola.

10. Nadpisywanie metod i adnotacja Override

Klasa potomna może dostarczyć własną wersję odziedziczonej metody instancji. Nazywamy to nadpisywaniem, po angielsku overriding. Adnotacja `@Override` pomaga kompilatorowi sprawdzić, czy naprawdę nadpisujemy metodę, a nie omyłkowo tworzymy inną przez literówkę.

```
class Animal {
    void sound() { System.out.println("Dźwięk zwierzęcia"); }
}
class Dog extends Animal {
    @Override
    void sound() { System.out.println("Hau!"); }
}

Animal animal = new Dog();
animal.sound();
```

```
Wynik:
Hau!
```

Mimo że zmienna ma typ `Animal`, odwołuje się do obiektu `Dog`. Przy wywołaniu nadpisanej metody wybierana jest wersja właściwa dla faktycznego obiektu. To ważny przykład **polimorfizmu**: kod może używać wspólnego typu bazowego, a różne obiekty reagują na tę samą metodę zgodnie ze swoją klasą.

11. Konstruktor klasy potomnej i super

Tworząc obiekt klasy potomnej, trzeba również zainicjalizować część pochodzącą z klasy bazowej. Konstruktor może wywołać konstruktor rodzica za pomocą `super(...)`. Jeżeli klasa bazowa nie ma dostępnego konstruktora bezargumentowego, trzeba wskazać odpowiedni konstruktor z argumentami.

```
class Vehicle {
    String brand;
    Vehicle(String brand) { this.brand = brand; }
}
class Car extends Vehicle {
    Car(String brand) { super(brand); }
}
```

`super(brand)` przekazuje napis do konstruktora `Vehicle`, który ustawia odziedziczone pole. Konstruktor potomka nie pomija inicjalizacji klasy bazowej. Warto zapamiętać, że tworzenie obiektu klasy potomnej rozpoczyna łańcuch wywołań konstruktorów.

12. Czas życia zmiennej i czas życia obiektu

Zmienne lokalne są związane z wykonaniem metody lub bloku. Obiekty tworzone przez `new` znajdują się w pamięci zarządzanej przez JVM, a ich los nie jest wprost taki sam jak los nazw zmiennych lokalnych. Obiekt może być dostępny przez inną referencję nawet po tym, gdy jedna zmienna przestanie istnieć.

```
StringBuilder first = new StringBuilder("tekst");
StringBuilder second = first;
first = null;
System.out.println(second.toString());
```

```
Wynik:
tekst
```

`first` i `second` początkowo wskazują na ten sam obiekt. Zmiana `first` na `null` nie niszczy obiektu i nie zmienia `second`. Pamięcią nieużywanych obiektów zarządza garbage collector, ale moment odzyskania pamięci nie jest czymś, co początkujący program powinien zakładać w swoim działaniu.

Przykład 1. Example01StaticInstance

Porównujemy pole powiązane z klasą z polem powiązanym z konkretnym obiektem.

```
package block6;

public class Example01StaticInstance {
    public static void main(String[] args) {
        Counter first = new Counter("A");
        Counter second = new Counter("B");

        System.out.println(first.getName());
        System.out.println(second.getName());
        System.out.println(Counter.getCreatedCount());
    }
}
```

Wynik:

```
A
B
2
```

- W zapisie `Counter first = new Counter("A");` słowo `new` tworzy obiekt; przypisanie pozwala korzystać z niego przez odpowiednią zmienną.
- W zapisie `Counter second = new Counter("B");` słowo `new` tworzy obiekt; przypisanie pozwala korzystać z niego przez odpowiednią zmienną.
- Instrukcja `System.out.println(first.getName());` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.
- Instrukcja `System.out.println(second.getName());` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.
- Instrukcja `System.out.println(Counter.getCreatedCount());` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.

Przykład 2. Counter

Sprawdzamy, jak obliczyć liczbę utworzonych obiektów przy użyciu współdzielonego pola.

```
package block6;

public class Counter {
    private static int createdCount = 0;
    private final String name;

    public Counter(String name) {
        this.name = name;
        createdCount++;
    }

    public String getName() {
        return name;
    }

    public static int getCreatedCount() {
        return createdCount;
    }
}
```

- Instrukcja `createdCount++;` zmienia wartość istniejącej zmiennej. Jej nowa wartość będzie użyta przy następnym sprawdzeniu warunku lub obliczeniu.
- Instrukcja `return name;` kończy bieżące wywołanie metody i przekazuje wynik do miejsca wywołania.
- Instrukcja `return createdCount;` kończy bieżące wywołanie metody i przekazuje wynik do miejsca wywołania.

Przykład 3. FinalDemo

Przekonujemy się, że `final` zabrania ponownego przypisania zmiennej, ale nie musi blokować zmiany zawartości obiektu.

```
package block6;

public class FinalDemo {
    public static void main(String[] args) {
        final int maxAttempts = 3;
        final StringBuilder builder = new StringBuilder("A");
        builder.append("B"); // dozwolone: obiekt może się zmienić

        System.out.println(maxAttempts);
        System.out.println(builder);

        // maxAttempts = 4; // błąd
        // builder = new StringBuilder("X"); // błąd
    }
}
```

Wynik:

```
3
AB
```

- Zapis `final int maxAttempts = 3;` tworzy zmienną `maxAttempts` typu `int` i nadaje jej wartość początkową. Późniejsze instrukcje mogą jej używać w obliczeniach lub warunkach.
- W zapisie `final StringBuilder builder = new StringBuilder("A");` słowo `new` tworzy obiekt; przypisanie pozwala korzystać z niego przez odpowiednią zmienną.
- Instrukcja `System.out.println(maxAttempts);` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.
- Instrukcja `System.out.println(builder);` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.

Przykład 4. ScopeLifetime

Porównujemy zakres zmiennych lokalnych z dostępnością pól i obiektów.

```
package block6;

public class ScopeLifetime {
    private int instanceField = 10;
    private static int classField = 20;

    void show() {
        int local = 30;
        if (local > 0) {
            int insideBlock = 40;
            System.out.println(instanceField + classField + local + insideBlock);
        }
        // insideBlock nie istnieje już w tym zakresie.
    }

    public static void main(String[] args) {
        ScopeLifetime demo = new ScopeLifetime();
        demo.show();
    }
}
```

Wynik:

```
100
```

- Zapis `int local = 30;` tworzy zmienną `local` typu `int` i nadaje jej wartość początkową. Późniejsze instrukcje mogą jej używać w obliczeniach lub warunkach.
- W zapisie `if (local > 0) {` program podejmuje decyzję na podstawie wartości logicznej. Instrukcje w bloku są wykonywane tylko wtedy, gdy wynik warunku jest `true`.
- Zapis `int insideBlock = 40;` tworzy zmienną `insideBlock` typu `int` i nadaje jej wartość początkową. Późniejsze instrukcje mogą jej używać w obliczeniach lub warunkach.
- Instrukcja `System.out.println(instanceField + classField + local + insideBlock);` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.
- W zapisie `ScopeLifetime demo = new ScopeLifetime();` słowo `new` tworzy obiekt; przypisanie pozwala korzystać z niego przez odpowiednią zmienną.

Przykład 5. Example02Inheritance

Tworzymy obiekt klasy potomnej i wywołujemy na nim metodę nadpisaną po klasie bazowej.

```
package block6;

public class Example02Inheritance {
    public static void main(String[] args) {
        Dog dog = new Dog("Reks");
        dog.makeSound();
        dog.fetch();

        Animal animalReference = dog;
        animalReference.makeSound(); // działa wersja nadpisana w Dog
    }
}
```

Wynik:

```
Reks: hau!
Reks aportuje.
Reks: hau!
```

- W zapisie `Dog dog = new Dog("Reks");` słowo `new` tworzy obiekt; przypisanie pozwala korzystać z niego przez odpowiednią zmienną.

Przykład 6. Animal

Przyglądamy się klasie bazowej, z której inne typy mogą dziedziczyć wspólne cechy i zachowania.

```
package block6;

public class Animal {
    protected String name;

    public Animal(String name) {
        this.name = name;
    }

    public void makeSound() {
        System.out.println("Nieznany dźwięk");
    }
}
```

- Instrukcja `System.out.println("Nieznany dźwięk");` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.

Przykład 7. Dog

Sprawdzamy, co klasa potomna przejmuje po rodzicu, a co definiuje samodzielnie.

```
package block6;

public class Dog extends Animal {
    public Dog(String name) {
        super(name);
    }

    @Override
    public void makeSound() {
        System.out.println(name + ": hau!");
    }

    public void fetch() {
        System.out.println(name + " aportuje.");
    }
}
```

- Instrukcja `System.out.println(name + ": hau!");` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.
- Instrukcja `System.out.println(name + " aportuje.");` wypisuje wartość lub napis w konsoli. `println` dodatkowo przechodzi do nowej linii, a `print` pozostaje w bieżącej linii.

Zadanie 8. Vehicle

Dodaj pole protected String brand, konstruktor ustawiający markę oraz metodę describe() zwracającą opis pojazdu. Klasa będzie nadklasą Car.

```
package block6;

public class Vehicle {
    // TODO
}
```

Zadanie 9. Car

Zmień deklarację na Car extends Vehicle. Dodaj pole mileage, wywołaj super(brand) w konstruktorze i nadpisz describe(), podając także przebieg.

```
package block6;

public class Car {
    // TODO: po uzupełnieniu zmień deklarację na "public class Car extends Vehicle"
}
```

13. Zadania - pola, static, final i dziedziczenie

Zadanie 1. Utwórz trzy obiekty klasy `Counter` i zwiększ wspólny statyczny licznik podczas konstruowania obiektu. Sprawdź, jaka liczba zostanie wypisana. Wyjaśnij różnicę względem zwykłego pola instancji.

Zadanie 2. Utwórz klasę `Car` z polami `brand` oraz `mileage`. Zmień `mileage` tylko dla jednego z dwóch samochodów i upewnij się, że drugi zachował własną wartość.

Zadanie 3. Napisz `final int MAX = 5;` i spróbuj przypisać `MAX = 6;`. Odczytaj komunikat kompilatora. Następnie sprawdź, czy można wywołać `append` na zmiennej `final StringBuilder`.

Zadanie 4. Utwórz klasę `Animal` z metodą `sound`, a następnie `Cat` oraz `Dog`, które nadpiszą tę metodę. Przechowaj obiekt psa w zmiennej typu `Animal` i sprawdź, którą metodę wykona JVM.

Zadanie 5. Napisz metodę, która deklaruje zmienną lokalną `value` wewnątrz `if`. Spróbuj odwołać się do tej nazwy poza blokiem. Wyjaśnij komunikat kompilatora.